Trabajo final

Centro de control de misiones espaciales

Programación Orientada a Objetos

Grado en Ingeniería Informática · Modalidad online · Curso 2026/27

Dr. D. Antonio Serrano Fernández
Universidad Católica San Antonio de Murcia

1. Resumen

El trabajo final consiste en desarrollar **en grupos de 3 personas** una aplicación Java que ayude a una agencia espacial, inspirada en la NASA, a planificar sus misiones: qué cohete se usa, qué astronautas viajan y qué cargas se lanzan, respetando las reglas de seguridad de cada misión.

La práctica se desarrolla en **tres entregas** que siguen el temario de la asignatura y sirven también de preparación para las pruebas de evaluación: la E1 trabaja los contenidos de la primera prueba parcial y la E2, los de la prueba final.

Entrega	Contenido	Peso
E1 · Diseño y núcleo del modelo	Diagrama de clases UML y código de las clases del modelo con los contenidos de los Temas 1 a 4	20 %
E2 · Evolución del modelo	El modelo completo con colecciones, interfaces, excepciones propias, patrones e informes (sin interfaz gráfica)	30 %
E3 · Aplicación final	Interfaz gráfica, pruebas unitarias, versión final del código y vídeo-memoria	50 %

La nota de la práctica es la suma ponderada de las tres entregas y supone el **40 % de la nota final** de la asignatura, según la Guía Docente. Las fechas límite de la inscripción de grupos y de cada entrega se publican en el Campus Virtual.

> **Antes de empezar:** lea completos los apartados 2 (normas) y 6 (variantes). La variante que corresponde a su grupo depende del número de grupo y cambia parte de los requisitos.

2. Normas generales

2.1. Grupos

- La práctica se realiza **obligatoriamente en grupos de 3 integrantes**.
- Cada grupo se inscribe en la hoja compartida publicada en el Campus Virtual **dentro del plazo de inscripción** indicado en el propio Campus. El número de grupo es el de la fila de inscripción y determina la variante (apartado 6).
- Cada integrante es **responsable de uno de los tres módulos** de la aplicación (apartado 4). Todos deben conocer el conjunto del trabajo y participar en las partes comunes.
- La nota es, en principio, la misma para los tres integrantes. El profesor puede **ajustarla individualmente** cuando las evidencias (historial de GitHub y vídeo-memoria) muestren una participación claramente desigual.

2.2. Entorno de trabajo

- Lenguaje Java (JDK 21 o superior) e IDE **Eclipse**, que es el entorno de referencia de la asignatura.
- Paquete raíz `ucam.poo.mision` (estructura sugerida en el anexo C).
- Documentación de todas las clases públicas con **Javadoc**.

2.3. Repositorio en GitHub

- Cada grupo crea un **repositorio privado** llamado `poo2627-grupoNN` (NN = número de grupo, con dos cifras) e invita al profesor como colaborador. El usuario de GitHub del profesor se indica en el Campus.
- El repositorio debe crearse al comenzar el trabajo, de modo que el código de la Entrega 1 se desarrolle ya en él. El anexo B resume los pasos básicos.

- Cada integrante sube su trabajo **con su propia cuenta** y con mensajes de commit descriptivos.
- Debe haber commits de **cada integrante en cada uno de los tres periodos**: hasta la E1, entre la E1 y la E2, y entre la E2 y la E3.
- Un historial con uno o pocos commits masivos al final del plazo, o con una aportación muy desequilibrada entre integrantes, se penalizará en el criterio de evidencias de trabajo (apartado 8) y puede dar lugar a notas individuales distintas.
- El repositorio no sustituye a la entrega en el Campus: el ZIP del Campus es la entrega oficial.

2.4. Uso de inteligencia artificial

> **Está prohibido el uso de herramientas de IA generativa** (ChatGPT, Gemini —incluido el Gemini integrado en el Campus—, Copilot, Claude u otras) para elaborar el código, el diseño o las explicaciones de la práctica y del vídeo-memoria. Solo se permite para revisar la redacción de textos escritos por el propio grupo.

Es una asignatura básica de programación: el objetivo es que cada estudiante aprenda a diseñar y programar por sí mismo. El uso de IA fuera de lo permitido se tratará como **copia** y se aplicará la normativa de la Universidad. Cada entrega incluye una declaración firmada por los tres integrantes (anexo D).

2.5. Copia y verificación de autoría

- La copia de prácticas, total o parcial, se penaliza según la normativa de la Universidad con el suspenso de todas las partes de la convocatoria, tanto para quien copia como para quien se deja copiar.
- Según la Guía Docente, el profesor se reserva el derecho de mantener una **videoconferencia privada** con el grupo o con cualquiera de sus integrantes para verificar la autoría. Una defensa inadecuada supone el suspenso de la práctica.
- Un código que no compila, o que no incluye todo lo necesario para compilar, se califica con un 0 en la entrega correspondiente.
- No se admiten entregas fuera de plazo.

3. Enunciado

La agencia espacial necesita una aplicación para su **Centro de Control de Misiones**. Hasta ahora planifica cada lanzamiento con hojas de cálculo y quiere un sistema que registre sus cohetes, sus astronautas y sus cargas, y que compruebe automáticamente que cada misión cumple las reglas de seguridad antes de autorizar el lanzamiento.

3.1. Datos que hay que modelar

Agencia espacial. La aplicación gestiona una única agencia, con nombre y sede. La agencia mantiene el registro de sus cohetes, astronautas, cargas útiles y misiones, y permite localizar cada elemento por su código.

Cohete. Tiene un código único (por ejemplo, `SLS-01`), un modelo, una capacidad máxima de carga en kg, un número de plazas para tripulación (0 si no es tripulado), si es reutilizable o no, y un estado: disponible, asignado a una misión, en mantenimiento o retirado.

Carga útil. Todo lo que viaja en una misión que no es tripulación. Tiene un identificador, un nombre y una masa en kg. Hay varios tipos de carga y cada uno calcula de forma distinta su **masa efectiva de lanzamiento**:

- **Satélite:** masa más el combustible de maniobra que lleva a bordo.
- **Sonda:** masa más el combustible necesario para su trayectoria. Además, tiene un destino (por ejemplo, Marte).

- **Módulo habitable:** masa más 250 kg de soporte vital por cada plaza del módulo.
- Un **cuarto tipo** que depende de la variante del grupo (apartado 6).

Astronauta. Tiene un identificador único (por ejemplo, `AST-014`), nombre, apellidos, año de nacimiento, nacionalidad, especialidad (comandante, piloto, ingeniero, médico o científico) y horas de vuelo acumuladas. Además:

- Puede tener **necesidades especiales** registradas (por ejemplo, dieta sin gluten o medicación específica).
- Puede tener **compañeros preferentes**, con los que ha entrenado y con los que prefiere volar.
- Puede tener **incompatibilidades**: astronautas con los que no puede compartir misión.

Hay tres tipos de astronauta y cada uno calcula de forma distinta su **experiencia computable**, que se muestra en el listado de astronautas:

- **De carrera:** horas de vuelo más 100 h por cada misión completada.
- **En formación:** horas de vuelo más la mitad de sus horas de simulador.
- **Invitado:** personal externo a la agencia, del que se registra la entidad que lo envía; solo cuentan sus horas de vuelo.

Misión. Tiene un código, un nombre, un tipo (orbital, estación espacial, lunar o marciana), una fecha prevista de lanzamiento, un cohete asignado, unas cargas y un estado (planificada, lista, lanzada, completada o cancelada). Cada misión tiene su propia **bitácora**, que se crea con la misión y registra con fecha y texto cada cambio relevante (por ejemplo, «tripulación asignada» o «misión lanzada»). Las misiones pueden ser tripuladas o no tripuladas, y calculan de forma distinta su **masa total de lanzamiento**:

- **No tripulada:** no lleva astronautas y se opera desde un centro de control en tierra, cuyo nombre se registra. Su masa total es la suma de las masas efectivas de sus cargas.
- **Tripulada:** lleva además una tripulación de astronautas. Su masa total es la suma de las masas efectivas de sus cargas más 120 kg por cada astronauta (tripulante y equipo personal).

3.2. Reglas de seguridad

Una misión solo puede pasar a estado **lista** y lanzarse si cumple todas estas reglas. Cuando una operación incumple alguna, la aplicación debe impedirla e informar del motivo; a partir de la E2, mediante **excepciones propias**.

1. La masa total de lanzamiento de la misión no supera la capacidad máxima de carga del cohete.
2. En una misión tripulada, el número de astronautas no supera las plazas del cohete.
3. Toda misión tripulada incluye al menos un comandante y un piloto.
4. No viajan juntos dos astronautas incompatibles.
5. El cohete está disponible: no está en mantenimiento, ni retirado, ni asignado a otra misión activa (planificada, lista o lanzada).
6. La regla adicional de la variante del grupo (apartado 6).

3.3. Funcionalidad mínima

El apartado 7 indica qué parte de esta funcionalidad se exige en cada entrega.

- **Gestión** de cohetes, cargas, astronautas y misiones: alta, baja, modificación y consulta.
- **Asignación manual** de cohete, tripulación y cargas a una misión, aplicando las reglas del apartado 3.2.
- **Lanzamiento y cierre** de misiones: una misión lista pasa a lanzada; al completarse, el cohete vuelve a estar disponible si es reutilizable y queda retirado si no lo es. Todo se anota en la bitácora.
- **Informes en ficheros de texto**:

  - Listado de astronautas, ordenado según el criterio de la variante.
  - Listado de misiones planificadas y listas, con su cohete, su tripulación, sus cargas y la masa total de lanzamiento.
- **Pruebas unitarias** con JUnit de las reglas de seguridad (solo en la E3): al menos dos pruebas por integrante.
- **Interfaz gráfica** con Swing (solo en la E3): ventana principal desde la que se puede, como mínimo, dar de alta y consultar astronautas, crear una misión y asignarle cohete y tripulación (mostrando los errores de validación en un diálogo) y generar los dos informes. El resto de operaciones pueden quedarse fuera de la interfaz gráfica.
- Un **programa de demostración** (`DemoConsola`) que cargue unos datos de ejemplo y ejercite los casos válidos y los que deben fallar.

3.4. Funcionalidad adicional (hasta +1 punto en la E3)

- **Asignación automática de tripulación** a una misión, con al menos dos algoritmos intercambiables en tiempo de ejecución. Por ejemplo, priorizar horas de vuelo o priorizar compañeros preferentes.
- Exportar alguno de los informes a CSV o PDF.
- Cualquier otra ampliación propuesta por el grupo y justificada en el vídeo.

4. Módulos y reparto

La aplicación se divide en tres módulos. Cada integrante es el responsable principal de uno de ellos: lo diseña, lo programa, escribe sus pruebas y lo explica en el vídeo. El reparto se declara en la E1 y no puede cambiarse después sin avisar al profesor.

Módulo	Contenido mínimo	Responsable
Flota	Cohetes y cargas útiles, incluida la carga de la variante; cálculo de la masa efectiva de lanzamiento.	Integrante 1
Tripulación	Astronautas y sus tipos: especialidad, experiencia computable, necesidades especiales, compañeros preferentes e incompatibilidades; criterio de ordenación de la variante.	Integrante 2
Misiones	Misiones tripuladas y no tripuladas y su bitácora; masa total de lanzamiento; reglas de seguridad, incluida la de la variante, y tratamiento de sus incumplimientos; lanzamiento y cierre.	Integrante 3
Partes comunes	Agencia espacial, generación de informes, interfaz gráfica (cada integrante programa al menos la parte de su módulo) y programa de demostración.	Los tres

5. Requisitos de diseño por tema

La práctica debe demostrar los contenidos de la asignatura. Esta tabla indica qué se exige como mínimo y en qué entrega se revisa por primera vez. Decidir qué clases, interfaces y relaciones concretas cumplen cada requisito forma parte del trabajo de diseño del grupo, que deberá justificar sus decisiones.

Tema	Qué debe demostrar el diseño	Desde
T2 · Clases	Atributos privados y métodos con criterio; constructores que dejan el objeto en un estado válido; constantes static final; al menos tres enumeraciones (enum).	E1

Tema	Qué debe demostrar el diseño	Desde
T3 · Relaciones	Asociaciones entre clases, con sus roles y multiplicidades; al menos una asociación de una clase consigo misma (reflexiva); al menos una composición; arrays de objetos en las relaciones con varios elementos. Las relaciones deben aparecer también en el diagrama de clases UML.	E1
T4 · Herencia	En cada módulo, al menos una jerarquía de herencia con una clase abstracta y un comportamiento polimórfico, programada por el integrante responsable del módulo; redefinición de toString, equals y hashCode en las clases en las que tenga sentido.	E1
T5 · Interfaces y colecciones	Al menos una interfaz propia, implementada por clases de módulos distintos y utilizada como tipo, de modo que un mismo código trate de forma uniforme objetos de clases diferentes. Uso justificado de List, Set y Map con tipos genéricos. Ordenación con Comparator.	E2
T6 · Patrones	Al menos dos patrones de diseño distintos de los estudiados en la asignatura, aplicados donde esté justificado.	E2
T7 · Robustez	Al menos dos excepciones propias, una de ellas comprobada (checked); ninguna excepción sin tratar llega al usuario. Pruebas JUnit de las reglas, en la E3.	E2
T8 · Swing	Interfaz gráfica mínima del apartado 3.3, con los errores mostrados en diálogos.	E3

> La escritura en ficheros de texto no forma parte del temario; el anexo A incluye un fragmento de partida. Es posible, además, que alguna otra parte del trabajo requiera consultar información más allá de lo explicado en la asignatura.

6. Variantes por grupo

Cada grupo tiene una variante que cambia tres requisitos. La variante se obtiene del número de grupo: grupos 1, 5, 9… → A; grupos 2, 6, 10… → B; grupos 3, 7, 11… → C; grupos 4, 8, 12… → D.

	Variante A	Variante B	Variante C	Variante D
Orden del listado de astronautas	Por apellidos y nombre	Por experiencia computable, de mayor a menor	Por especialidad y, dentro de cada una, por apellidos	Por año de nacimiento, de más joven a más veterano
Regla adicional de seguridad	Toda misión tripulada incluye al menos un médico	Al menos la mitad de la tripulación tiene 1000 h de experiencia computable o más	Las misiones lunares y marcianas llevan tripulantes de al menos dos nacionalidades	Un astronauta no puede estar en dos misiones planificadas o listas a la vez

	Variante A	Variante B	Variante C	Variante D
Cuarto tipo de carga	Rover: masa + baterías (kg)	Telescopio: masa + escudo térmico (kg)	Suministros: masa × 1,15 (embalaje)	Cápsula de retorno: masa + escudo térmico + paracaídas (kg)

En la E1 debe indicarse la variante del grupo y dónde aparece en el diseño; el orden del listado se exige a partir de la E2. Una práctica que implemente otra variante se considerará copiada.

7. Entregas

En cada entrega sube los ficheros **un solo integrante** del grupo, en la tarea correspondiente del Campus Virtual, donde se indica la fecha límite. La memoria deberá incluir en la portada el nombre de todos los integrantes del grupo. Todos los ficheros llevan el prefijo `GrupoNN`.

7.1. Entrega 1 · Diseño y núcleo del modelo (20 %)

Se entrega en dos tareas del Campus:

- **Código:** `GrupoNN_E1.zip` con el proyecto Eclipse, que debe compilar sin errores.
- **Memoria:** `GrupoNN_E1.pdf`.

Alcance del código:

- Las clases del modelo de los tres módulos, con sus atributos, constructores, enumeraciones y relaciones, incluida la jerarquía de herencia de cada módulo.
- La asignación de cohete, tripulación y cargas a una misión, con la comprobación de las reglas de seguridad, y el lanzamiento y cierre de misiones con su bitácora.
- La clase `DemoConsola`, con unos datos de ejemplo, casos válidos y casos rechazados por las reglas.

En esta entrega el código debe limitarse a los contenidos de los Temas 1 a 4: las relaciones con varios objetos se implementan con arrays, y una operación que incumple una regla no se realiza e informa de ello sin utilizar excepciones. No se utilizan todavía colecciones, interfaces propias, excepciones propias ni patrones de diseño, y tampoco se piden la gestión desde la agencia espacial ni los informes.

Contenido de la memoria:

1. Integrantes, módulo del que es responsable cada uno y variante del grupo.
2. Requisitos de la aplicación con palabras propias del grupo: qué hace y qué reglas cumple (máximo una página).
3. Diagrama de clases UML del modelo, coherente con el código entregado: clases, atributos y métodos principales con su visibilidad, relaciones con sus roles y multiplicidades, y herencia. Puede utilizarse Visual Paradigm, ArgoUML u otra herramienta.
4. Justificación breve de las decisiones de diseño más importantes (máximo una página).
5. Enlace al repositorio de GitHub, con el código de la entrega y el profesor ya invitado.
6. Declaración de no uso de IA (anexo D) firmada por los tres integrantes.

7.2. Entrega 2 · Evolución del modelo (30 %)

Qué se entrega: `GrupoNN_E2.zip` con el proyecto Eclipse completo, que debe compilar sin errores. Debe incluir:

- El modelo de la E1 reescrito con los contenidos de los Temas 5 a 7: colecciones en lugar de arrays, excepciones propias para los incumplimientos de las reglas, al menos una interfaz propia y al menos dos patrones de diseño.

- La agencia espacial, con la gestión de cohetes, cargas, astronautas y misiones, y la generación de informes.
- La clase `DemoConsola` actualizada.
- La documentación Javadoc generada.
- Un fichero `CAMBIOS.md` que explique qué ha cambiado respecto a la E1 y por qué: qué estructura o mecanismo sustituye a cuál y qué cambios responden a los comentarios del profesor.
- El enlace al repositorio y la declaración de no uso de IA, en un fichero `LEEME.txt`.

En esta entrega no se piden interfaz gráfica ni pruebas unitarias.

7.3. Entrega 3 · Aplicación final y vídeo-memoria (50 %)

Se entrega en dos tareas del Campus:

- **Código:** `GrupoNN_E3.zip` con el proyecto Eclipse final (incluida la interfaz gráfica), las pruebas JUnit (al menos dos por integrante), el Javadoc y el `CAMBIOS.md` actualizado.
- **Vídeo-memoria:** `GrupoNN_E3.pdf` de 1–2 páginas con el enlace al vídeo, el diagrama UML final, la tabla de reparto y la declaración de no uso de IA.

Requisitos del vídeo-memoria:

- Duración máxima de **10 minutos**. Alojado en YouTube (oculto, es decir, no listado), con acceso para el profesor.
- **Cada integrante explica su propio módulo durante unos 3 minutos**, con el código en pantalla y sin leer un guion. Debe identificarse al empezar su parte, preferiblemente con la cámara activada.
- Cada integrante explica también **una decisión de diseño** que tomó y **un error** que encontró y cómo lo resolvió.
- El vídeo incluye una demostración de la aplicación en funcionamiento: un caso válido y un caso rechazado por una regla de seguridad.

8. Evaluación

Cada entrega se califica sobre 10 con las rúbricas siguientes. La nota de la práctica es `0,20·E1 + 0,30·E2 + 0,50·E3`.

8.1. Rúbrica de la E1

Criterio	Puntos
Diseño en UML: clases, relaciones, multiplicidades y herencia	3
Código: encapsulación, relaciones y jerarquías con polimorfismo	3
Reglas de seguridad y demostración por consola funcionando	2
Variante aplicada y decisiones justificadas	1
Commits de todos los integrantes en el periodo y reparto declarado	1

8.2. Rúbrica de la E2

Criterio	Puntos
Modelo completo, gestión desde la agencia y reglas de seguridad funcionando	3
Uso correcto de la POO: encapsulación, herencia y polimorfismo, interfaz, colecciones y genéricos	3
Excepciones propias, informes en ficheros y patrones de diseño aplicados	2

Criterio	Puntos
Javadoc y programa de demostración	1
CAMBIOS.md y commits de todos los integrantes en el periodo	1

8.3. Rúbrica de la E3

Criterio	Puntos
Funcionalidad mínima completa, incluida la interfaz gráfica	2,5
Calidad del diseño: responsabilidades, cohesión, acoplamiento y patrones	2
Control de errores: excepciones y mensajes en la interfaz	1,5
Pruebas y documentación	1
Vídeo-memoria: explicación de cada integrante, decisiones y demostración	2
Evidencias de trabajo continuo y equilibrado en GitHub	1
Funcionalidad adicional (apartado 3.4)	hasta +1

Anexo A. Escribir un fichero de texto

Este fragmento escribe una lista de líneas en un fichero de texto y sirve de punto de partida para generar los informes; el grupo debe adaptarlo a su diseño. La estructura try (...) con recursos cierra el fichero automáticamente; `IOException` es una excepción comprobada que hay que tratar (Tema 7).

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;

public void escribirFichero(String fichero, List<String> lineas) {
    try (PrintWriter salida = new PrintWriter(new FileWriter(fichero))) {
        for (String linea : lineas) {
            salida.println(linea);
        }
    } catch (IOException e) {
        System.err.println("No se pudo escribir " + fichero
                + ": " + e.getMessage());
    }
}
```

En la E3, el mensaje de error debe mostrarse también en la interfaz gráfica, no solo por consola.

Anexo B. GitHub en cinco pasos

1. Un integrante crea el repositorio privado `poo2627-grupoNN` en github.com e invita a sus compañeros y al profesor (Settings → Collaborators).
2. Cada integrante lo clona en Eclipse: File → Import → Git → Projects from Git → Clone URI.
3. Para guardar un avance: botón derecho sobre el proyecto → Team → Commit…; se seleccionan los ficheros, se escribe un mensaje descriptivo y se pulsa Commit and Push.
4. Antes de empezar a trabajar cada día: Team → Pull, para traer los cambios de los compañeros.
5. Para evitar conflictos, cada integrante trabaja sobre todo en los ficheros de su módulo y se coordina con el grupo antes de modificar las partes comunes.

Alternativa sin Eclipse: GitHub Desktop (desktop.github.com), con las mismas operaciones de commit, push y pull.

Anexo C. Estructura de paquetes sugerida

Paquete	Contenido
`ucam.poo.mision.flota`	Clases del módulo Flota
`ucam.poo.mision.tripulacion`	Clases del módulo Tripulación
`ucam.poo.mision.misiones`	Clases del módulo Misiones
`ucam.poo.mision.comun`	Partes comunes del modelo
`ucam.poo.mision.gui`	Ventanas y diálogos de Swing
`ucam.poo.mision.app`	DemoConsola y la clase con el main de la aplicación gráfica

Anexo D. Declaración de autoría y de no uso de IA

Este texto debe incluirse en cada entrega, firmado por los tres integrantes:

> «Los abajo firmantes, integrantes del grupo NN, declaramos que el código, el diseño y las explicaciones de esta entrega son obra exclusiva nuestra, que no hemos utilizado herramientas de inteligencia artificial generativa para elaborarlos y que solo las hemos usado, en su caso, para revisar la redacción. Conocemos que el incumplimiento de esta declaración se considerará copia a efectos de la normativa de la Universidad.»

Nombre, apellidos, DNI y firma de cada integrante.